

ICPC Team Notebook

typedef unsigned long long ll;
Sorbonne Université
SWERC 2025

A curated reference of algorithms and data structures



November 19, 2025

Contents

1	Data Structures	1
1.1	Fenwick Tree	1
1.2	RMQ	1
1.3	Segment Tree(s)	1
1.4	Trie	2
1.5	Union find	2
2	Graph Algorithms	2
2.1	2-SAT	2
2.2	Articulation Points and Bridges	2
2.3	Cycle detection	3
2.4	Euler paths	3
2.5	Hungarian	3
2.6	Iterative dfs	4
2.7	LCA	4
2.8	Max Bipartite Matching	4
2.9	Max Flow Min Cut	4
2.10	Min Cost Flow	5
2.11	Minimum Spanning Tree	5
2.12	Shortest Path	6
2.13	Strongly Connected Components	6
2.14	Toposort	6
3	Dynamic Programming	6
3.1	Coin Change	6
3.2	Convex Hull Trick	7
3.3	Kadane	7
3.4	Knapsack	7
3.5	Longest Increasing Subsequence	7
3.6	MinPathSum	7
3.7	TSP	8
4	Strings	8
4.1	Aho-Corasick	8
4.2	Knuth-Morris-Pratt	8
4.3	Manacher	8
4.4	SuffixArray	9
5	Arrays	9
5.1	Inversions	9
6	Math	9
6.1	Fourier Transform	9
6.2	Linear Algebra	10
6.3	Matrix Operations	10
6.4	Number Theory	11
6.5	Useful NT Functions in Python	11
6.6	Useful NT Functions in c++	11
7	Geometry	12
7.1	Circles	12
7.2	Closest Points	12
7.3	Points and Lines	12
7.4	Polygons	13
7.5	Useful Theory	14
8	Triangles	14
9	Quadrilatères	14
9.1	Quadrilatères cycliques (inscriptibles)	14

10	Coordonnées sphériques	14
11	Utilities	14
11.1	STL	14
1	Data Structures	1
1.1	Fenwick Tree	2
Point update, prefix and range sum, $O(\log n)$, $O(n)$ build		
<pre>struct Fenwick { int n; vector<ll> t; //using long long, int might overflow. Fenwick(vector<ll>& a): n(a.size()), t(n+1,0) { for(int i = 1; i <=n; i++) { // Builds tree from array in O(n) t[i] += a[i-1]; int p = i+(i&-i); if(p<=n) t[p] += t[i]; } } void add(int i,long long v){ for(;i<=n;i+=i&-i) t[i]+=v; } long long sum(int i){ long long r=0; for(; i>0 ;i -= i&-i) r += t[i]; return r; } long long sum(int l,int r){ return sum(r)-sum(l-1); } };</pre>		
1.2	RMQ	7
Preprocess an array in $O(n \log n)$ to answer range minimum queries in $O(1)$		
<pre>struct RMQ { vvi jmp; RMQ(const vi& V) { // Python: depth = N.bit_length() int N = V.size(), depth = 32 - __builtin_clz(N); jmp.assign(depth, V); for(int i = 0; i < depth-1; ++i) for(int j = 0; j < N; ++j) jmp[i+1][j] = min(jmp[i][j], jmp[i][min(N-1, j + (1<<i))]); } // Returns min{V[a], V[a+1], ..., V[b-1]} int query(int a, int b) { if(b <= a) return INT_MAX; int dep = 31 - __builtin_clz(b - a); // python: (b-a).bit_length() - 1 return min(jmp[dep][a], jmp[dep][b - (1<<dep)]); } };</pre>		
1.3	Segment Tree(s)	11
Regular seg tree: point update, range sum query, $O(\log n)$		
<pre>struct SegTree { int n; vector<ll> t; SegTree(vi &a): n(a.size()), t(4*n) { build(a,1,0,n); } void build(vi &a,int v,int l,int r){ if(r-l == 1){ t[v]=a[l]; return; } int m = (l+r)/2; build(a, 2*v, l, m); build(a, 2*v+1, m, r); t[v] = t[2*v] + t[2*v+1]; } ll sum_aux(int v,int l,int r,int ql,int qr){ if(qr <= l r <= ql) return 0; if(ql <= l && r <= qr) return t[v];</pre>		

```

    int m = (l+r)/2;
    return sum_aux(2*v, l,m,ql,qr) + sum_aux(2*v+1, m,r,ql,qr);
}
ll sum(int ql,int qr){ return sum_aux(1,0,n,ql,qr); }
void upd_aux(int v,int l,int r,int pos,ll val){
    if(r-l==1){ t[v]=val; return; }
    int m = (l+r)/2;
    if(pos<m) upd_aux(2*v,l,m,pos,val);
    else upd_aux(2*v+1,m,r,pos,val);
    t[v]=t[2*v]+t[2*v+1];
}
void upd(int pos,ll val){ upd_aux(1,0,n,pos,val); }
};

```

Lazy SegTree: range add, range sum query, $O(\log n)$

```

struct LazySegTree {
    int n; vector<ll> t, lazy;
    LazySegTree(vi &a):n(a.size()),t(4*n),lazy(4*n){build(a,1,0,n);}
    void build(vi &a,int v,int l,int r){
        if(r-l == 1){ t[v]=a[l]; return; }
        int m = (l+r)/2;
        build(a,2*v,l,m); build(a,2*v+1,m,r);
        t[v] = t[2*v]+t[2*v+1];
    }
    void push(int v,int l,int r){
        if(lazy[v]==0) return;
        t[v] += lazy[v]*(r-l);
        if(r-l>1){
            lazy[2*v] += lazy[v];
            lazy[2*v+1] += lazy[v];
        }
        lazy[v]=0;
    }
    void upd_aux(int v,int l,int r,int ql,int qr,ll val){
        push(v,l,r);
        if(qr<=l || r<=ql) return;
        if(ql<=l && r<=qr){
            lazy[v] += val; push(v,l,r);
            return;
        }
        int m = (l+r)/2;
        upd_aux(2*v, l,m,ql,qr,val);
        upd_aux(2*v+1, m,r,ql,qr,val);
        t[v] = t[2*v] + t[2*v+1];
    }
    ll sum_aux(int v,int l,int r,int ql,int qr){
        push(v,l,r);
        if(qr<=l || r<=ql) return 0;
        if(ql<=l && r<=qr) return t[v];
        int m = (l+r)/2;
        return sum_aux(2*v, l,m,ql,qr) + sum_aux(2*v+1, m,r,ql,qr);
    }
    void upd(int ql,int qr,ll val){ upd_aux(1,0,n,ql,qr,val); }
    ll sum(int ql,int qr){ return sum_aux(1,0,n,ql,qr); }
};

```

Iterative SegTree: point update, range sum query, $O(\log n)$

```

struct IterSegTree {
    int n; vector<ll> t;
    IterSegTree(vi &a): n(a.size()), t(2*n) {
        for(int i=0;i<n;i++) t[n+i]=a[i];
        for(int i=n-1;i>0;i--) t[i]=t[i<<1]+t[i<<1|1];
    }
    void upd(int pos,ll val){
        pos+=n; t[pos]=val;
    }
};

```

```

        for(pos>=1; pos>0; pos>=1) t[pos]=t[pos<<1]+t[pos<<1|1];
    }
    ll sum(int l,int r){
        ll res=0;
        for(l+=n,r+=n;l<r;l>=1,r>=1){
            if(l&1) res+=t[l++];
            if(r&1) res+=t[--r];
        }
        return res;
    }
};

```

1.4 Trie

Prefix tree: stores strings, insert/find in $O(|s|)$; handles lowercase letters a-z

```

struct Node{
    int next[26]; bool isEnd=false;
    Node(){fill(begin(next), end(next), -1);}
};
struct Trie{
    vector<Node> t;
    Trie() {t.emplace_back();}
    void insert(string s){
        int cur = 0;
        for (auto c : s){
            if (t[cur].next[c-'a'] == -1){
                t.emplace_back(); // Adds new node
                t[cur].next[c-'a'] = t.size() - 1;
            }
            cur = t[cur].next[c-'a'];
        }
        t[cur].isEnd = true;
    }
    bool find(string s){
        int cur = 0;
        for (auto c : s){
            if (t[cur].next[c-'a'] == -1) return false;
            cur = t[cur].next[c-'a'];
        }
        return t[cur].isEnd;
    }
};

```

1.5 Union find

Complexity: effectively $O(1)$

```

struct union_find{
    vector<int> rank, parent;
    union_find(int n){
        rank.resize(n, 0); parent.resize(n);
        for (int i = 0; i < n; i++) parent[i] = i;
    }
    int find(int i){
        int root = parent[i];
        if (parent[root] != root) return parent[i] = find(root);
        return root;
    }
    void unite(int x, int y) {
        int xRoot = find(x);
        int yRoot = find(y);
        if (xRoot == yRoot) return;
        if (rank[xRoot] < rank[yRoot]) parent[xRoot] = yRoot;
        else if (rank[yRoot] < rank[xRoot]) parent[yRoot] = xRoot;
    }
};

```

```

    else{
        parent[yRoot] = xRoot;
        rank[xRoot]++;
    }
}
};

```

2 Graph Algorithms

2.1 2-SAT

Check if n variables satisfy $(x_i \vee x_j) \wedge (\neg x_k \vee x_l) \wedge \dots$

```

struct TwoSAT {
    int n; SCC g; // c.f. strongly connected components
    TwoSAT(int vars): n(vars), g(2*vars) {}
    int var(int i){ return i; }
    int neg(int i){ return i+n; }
    void add_clause(int a, bool a_val, int b, bool b_val){
        int u = a_val ? var(a) : neg(a);
        int v = b_val ? var(b) : neg(b);
        g.add_edge(u^1, v); //  $\neg a \rightarrow b$ .  $\wedge$  bitwise XOR, same in python
        g.add_edge(v^1, u); //  $\neg b \rightarrow a$ 
    }
    bool satisfiable(vi& ans){
        g.kosaraju();
        ans.assign(n,0);
        for(int i=0;i<n;i++){
            if(g.scc[var(i)] == g.scc[neg(i)]) return false;
            ans[i] = g.scc[var(i)] > g.scc[neg(i)];
        }
        return true;
    }
};

```

2.2 Articulation Points and Bridges

$O(V + E)$

AP = vertices whose removal increases # of connected components.

Bridge = edges whose removal increases # of connected components.

```

void tarjan(int u, vvi &adj, vi &vis, vi &disc, vi &low,
            int &time, int p, vi &isAP, set<pii> &bridges) {
    vis[u] = 1;
    disc[u] = low[u] = ++time;
    int children = 0;
    for (int v : adj[u]) {
        if (v == p) continue;
        if (!vis[v]) {
            children++;
            tarjan(v, adj, vis, disc, low, time, u, isAP, bridges);
            low[u] = min(low[u], low[v]);
            if (low[v] > disc[u]) // Bridge
                bridges.insert({min(u, v), max(u, v)});
            if (p != -1 && low[v] >= disc[u])
                isAP[u] = 1; // Articulation Point
        }
        else low[u] = min(low[u], disc[v]);
    }
    if (p == -1 && children > 1)
        isAP[u] = 1;
}

pair<vi, set<pii>> findAPandBridges(int V, vvi &adj) {
    vi disc(V), low(V), vis(V), isAP(V); // Initialize with 0s
    set<pii> bridges;
    int time = 0;
}

```

```

for (int i = 0; i < V; i++)
    if (!vis[i])
        tarjan(i, adj, vis, disc, low, time, -1, isAP, bridges);
return {isAP, bridges};
}

```

2.3 Cycle detection

Returns a cycle in a directed graph, or empty if none exists. $O(n + m)$

```

vi dfs_cycle(const vvi &adj) {
    int n = adj.size();
    vi color(n, 0), path;
    vi cycle;
    function<bool(int)> dfs = [&](int v) {
        color[v] = 1; path.push_back(v);
        for (int u : adj[v]) {
            if (color[u] == 0 && dfs(u)) return true;
            else if (color[u] == 1) {
                auto it = find(path.begin(), path.end(), u);
                cycle.assign(it, path.end());
                return true;
            }
        }
        path.pop_back(); color[v] = 2;
        return false;
    };
    for (int i = 0; i < n; i++)
        if (color[i] == 0 && dfs(i)) break;
    return cycle;
}

```

Check if undirected graph contains a cycle in $O(m)$.

```

bool dsu_cycle(int n, vvi &edges) {
    union_find uf(n); //c.f. union find
    for (auto &e : edges) {
        if (uf.find(e[0]) == uf.find(e[1])) return true;
        uf.unite(e[0], e[1]);
    }
    return false;
}

```

Tortoise & Hare: finds start of cycle in a functional graph (-1 if none). $O(n)$

```

int floyd_cycle(int start, vi& nxt){
    int tort=start, hare=start;
    do {
        if (hare==-1 || nxt[hare]==-1) return -1;
        tort = nxt[tort];
        hare = nxt[nxt[hare]];
    } while (tort!=hare);
    tort=start;
    while (tort!=hare){
        tort=nxt[tort]; hare=nxt[hare];
    }
    return tort;
}

```

2.4 Euler paths

Euler Path/Circuits:

Undirected:

- Circuit: all vertices even degree and connected.

- Path: exactly 2 odd-degree vertices and connected; start at one of them.

- Trick: to use circuit function for path, add dummy edge between odd vertices, find circuit, remove dummy.

Directed:

- Circuit: strongly connected, $\text{indegree}[v] == \text{outdegree}[v] \forall v$.
- Path: one vertex with $\text{out-in}=1$ (start), one with $\text{in-out}=1$ (end), rest balanced.
- Trick: add dummy edge end- $\rightarrow$ start, find circuit, rotate to remove dummy.

Find Euler path in directed graph. $O(E)$ MODIFIES THE GRAPH!!!

```

vi euler_directed(vvi &adj, int start=0) {
    vi path; stack<int> st; st.push(start);
    while(!st.empty()) {
        int v = st.top();
        if(!adj[v].empty()) {
            int u = adj[v].back(); adj[v].pop_back();
            st.push(u);
        } else {
            path.push_back(v);
            st.pop();
        }
    }
    reverse(path.begin(), path.end());
    return path; // size = E+1 if valid Euler tour
}

```

Input: $\text{adj}[u][i] = v, \text{id}$, $\text{adj}[v][j] = u, \text{id}$ if u and v adjacent (same id for both edges).

```

vi euler_undirected(int n, vector<vpi>& adj) {
    vi used, path; stack<int> st; st.push(0);
    int m=0; for (auto &v:adj) m += v.size(); used.assign(m/2,0);
    while(!st.empty()){
        int v=st.top();
        while(!adj[v].empty() && used[adj[v].back().second])
            adj[v].pop_back();
        if(adj[v].empty()){
            path.push_back(v);
            st.pop();
        } else {
            auto e=adj[v].back(); adj[v].pop_back();
            used[e.second]=1;
            st.push(e.first);
        }
    }
    reverse(path.begin(), path.end());
    return path;
}

```

2.5 Hungarian

Min cost assignment $O(n^3)$. Input cost matrix, output (min cost, assignment)

```

pair<int, vi> hungarian(const vvi &a) {
    int n = a.size(), m = a[0].size();
    vi u(n+1), v(m+1), p(m+1), way(m+1);
    for (int i = 1; i <= n; i++) {
        p[0] = i;
        vi minv(m+1, 1e9); // INF constant
        vector<bool> used(m+1, false);
        int j0 = 0;
        do {
            used[j0] = true;
            int i0 = p[j0], delta = 1e9, j1 = 0;
            for (int j = 1; j <= m; j++) if (!used[j]) {
                int cur = a[i0-1][j-1] - u[i0] - v[j];
                if (cur < minv[j]) minv[j] = cur, way[j] = j0;
                if (minv[j] < delta) delta = minv[j], j1 = j;
            }
            for (int j = 0; j <= m; j++) {
                if (used[j]) u[p[j]] += delta, v[j] -= delta;
                else minv[j] -= delta;
            }
        } while (delta > 0);
    }
}

```

```

    }
    j0 = j1;
} while (p[j0] != 0);
do {
    int j1 = way[j0];
    p[j0] = p[j1];
    j0 = j1;
} while (j0);
}
vi match(n);
for (int j = 1; j <= m; j++) if (p[j]) match[p[j] - 1] = j - 1;
return {-v[0], match};
}

```

2.6 Iterative dfs

Iterative depth-first search

```

void dfs_iter(int start, vvi& adj, vi &output, vi& visited) {
    stack<int> st; st.push(start); visited[start] = 1;
    while (!st.empty()) {
        int v = st.top(); bool done = true;
        for (auto u : adj[v]) {
            if (!visited[u]) {
                visited[u] = 1; st.push(u);
                done = false; break;
            }
        }
        if (done) {
            st.pop(); output.push_back(v);
        }
    }
}

```

2.7 LCA

```

struct LCA {
    int n; vvi adj;
    vi depth, first, euler;
    RMQ rmq; // c.f. RMQ structure.
    LCA(int _n) : n(_n), adj(n), depth(n), first(n), rmq(vi()) {}
    void add_edge(int u, int v) { adj[u].push_back(v); adj[v].push_back(u); }
    void dfs(int v, int p, int d) {
        first[v] = euler.size();
        depth[v] = d;
        euler.push_back(v);
        for (int u : adj[v]) if (u != p) {
            dfs(u, v, d+1);
            euler.push_back(v);
        }
    }
    void build(int root=0) {
        euler.clear();
        dfs(root, -1, 0);
        vi D(euler.size());
        for (int i=0; i<euler.size(); i++) D[i]=depth[euler[i]];
        rmq = RMQ(D);
    } // dist(a,b) = depth[a] + depth[b] - 2 * depth[lca];
    int query(int a, int b) {
        int l = first[a], r = first[b];
        if (l > r) swap(l, r);
        return euler[rmq.query(l, r+1)];
    }
};

```

2.8 Max Bipartite Matching

König's theorem: In a bipartite graph, the number of edges in a maximum matching equals the number of vertices in a minimum vertex cover.

Hopcroft–Karp $O(E\sqrt{V})$ left $[0..nL-1]$, right $[0..nR-1]$ adj from left to right.

```
int nL, nR;
vector<vi>adj;
vi dist, matchL, matchR;
bool bfs() {
    queue<int> q;
    dist.assign(nL, -1);
    for (int u = 0; u < nL; ++u)
        if (matchL[u] == -1) dist[u] = 0, q.push(u);
    bool found = 0;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : adj[u]) {
            int mu = matchR[v];
            if (mu == -1) found = 1;
            else if (dist[mu] == -1) dist[mu] = dist[u] + 1, q.push(mu);
        }
    }
    return found;
}
bool dfs(int u) {
    for (int v : adj[u]) {
        int mu = matchR[v];
        if (mu == -1 || (dist[mu] == dist[u] + 1 && dfs(mu))) {
            matchL[u] = v; matchR[v] = u;
            return 1;
        }
    }
    dist[u] = -1;
    return 0;
}
int hopcroftKarp() {
    matchL.assign(nL, -1);
    matchR.assign(nR, -1);
    int matching = 0;
    while (bfs())
        for (int u = 0; u < nL; ++u)
            if (matchL[u] == -1 && dfs(u))
                ++matching;
    return matching;
}
```

2.9 Max Flow Min Cut

Dinic's max flow $O(V^2E)$, $O(E\sqrt{V})$ for bipartite/unit. Edges from reachable nodes after flow form a min cut.

```
struct FlowEdge {
    int v, u;
    long long cap, flow = 0;
    FlowEdge(int v, int u, long long cap) : v(v), u(u), cap(cap) {}
};
struct Dinic {
    const long long flow_inf = 1e18;
    vector<FlowEdge> edges;
    vector<vi> adj;
    int n, m = 0;
    int s, t; // source, target
    vi level, ptr;
    queue<int> q;
    Dinic(int n, int s, int t) : n(n), s(s), t(t) {
```

```
        adj.resize(n); level.resize(n); ptr.resize(n);
    }
    void add_edge(int v, int u, long long cap) {
        edges.emplace_back(v, u, cap);
        edges.emplace_back(u, v, 0);
        adj[v].push_back(m);
        adj[u].push_back(m + 1);
        m += 2;
    }
    bool bfs() {
        while (!q.empty()) {
            int v = q.front(); q.pop();
            for (int id : adj[v]) {
                if (edges[id].cap == edges[id].flow) continue;
                if (level[edges[id].u] != -1) continue;
                level[edges[id].u] = level[v] + 1;
                q.push(edges[id].u);
            }
        }
        return level[t] != -1;
    }
    long long dfs(int v, long long pushed) {
        if (pushed == 0) return 0;
        if (v == t) return pushed;
        for (int& cid = ptr[v]; cid < (int)adj[v].size(); cid++) {
            int id = adj[v][cid]; int u = edges[id].u;
            if (level[v] + 1 != level[u]) continue;
            long long tr = dfs(u, min(pushed, edges[id].cap - edges[id].flow));
            if (tr == 0) continue;
            edges[id].flow += tr;
            edges[id ^ 1].flow -= tr;
            return tr;
        }
        return 0;
    }
    long long flow() {
        long long f = 0;
        while (true) {
            fill(level.begin(), level.end(), -1);
            level[s] = 0; q.push(s);
            if (!bfs()) break;
            fill(ptr.begin(), ptr.end(), 0);
            while (long long pushed = dfs(s, flow_inf))
                f += pushed;
        }
        return f;
    }
    vector<pii> min_cut_edges() {
        vector<bool> vis(n, false);
        queue<int> q;
        q.push(s); vis[s] = true;
        while (!q.empty()) {
            int v = q.front(); q.pop();
            for (int id : adj[v]) {
                auto &e = edges[id];
                if (!vis[e.u] && e.cap > e.flow) {
                    vis[e.u] = true;
                    q.push(e.u);
                }
            }
        }
        vector<pii> cut;
        for (auto &e : edges) {
            if (vis[e.v] && !vis[e.u] && e.cap > 0) {
```

```

        cut.push_back({e.v, e.u});
    }
}
return cut;
}
};

```

2.10 Min Cost Flow

Finds min cost to send up to F flow (set $F=\text{INF}$ for MCMF) $O(FE \log V)$. Handles neg costs (no neg cycles).

```

struct MCF {
    struct E { int v, rev, cap; ll cost; };
    int n; vector<vector<E>> g;
    MCF(int n): n(n), g(n) {}
    void addEdge(int u,int v,int cap,ll cost){
        g[u].push_back({v,(int)g[v].size(),cap,cost});
        g[v].push_back({u,(int)g[u].size()-1,0,-cost});
    }
    pair<ll,ll> minCostFlow(int s,int t,int maxf){
        ll flow=0,cost=0; vll pot(n),dist(n); vi pv(n),pe(n);
        while(flow<maxf){
            fill(dist.begin(),dist.end(),LLONG_MAX);
            priority_queue<pii,vector<pii>,greater<>> pq;
            dist[s]=0; pq.push({0,s});
            while(!pq.empty()){
                auto [d,u]=pq.top(); pq.pop();
                if(d!=dist[u]) continue;
                for(int i=0;i<(int)g[u].size();i++){
                    E &e=g[u][i]; if(!e.cap) continue;
                    ll nd=d+e.cost+pot[u]-pot[e.v];
                    if(nd<dist[e.v]){
                        dist[e.v]=nd; pv[e.v]=u; pe[e.v]=i;
                        pq.push({nd,e.v});
                    }
                }
            }
            if(dist[t]==LLONG_MAX) break;
            for(int i=0;i<n;i++) if(dist[i]<LLONG_MAX) pot[i]+=dist[i];
            int add=maxf-flow;
            for(int v=t;v!=s;v=pv[v])
                add=min(add,g[pv[v]][pe[v]].cap);
            for(int v=t;v!=s;v=pv[v]){
                E &e=g[pv[v]][pe[v]];
                e.cap-=add; g[v][e.rev].cap+=add;
                cost+=add*e.cost;
            }
            flow+=add;
        }
        return {flow,cost};
    }
};

```

2.11 Minimum Spanning Tree

Kruskal $O(E \log E)$ edges list of $\{u,v,\text{weight}\}$

```

int kruskal(int V, vvi &edges) {
    sort(edges.begin(), edges.end(), [](auto a, auto b){return a[2]<b[2];});
    union_find uf(V);
    int cost = 0, count = 0;
    for (auto &e : edges) {
        if (uf.find(e[0]) != uf.find(e[1])) {
            uf.unite(e[0], e[1]);
            cost += e[2];
        }
    }
}

```

```

        if (++count == V - 1) break;
    }
}
return cost;
}

Prim's  $O((E + V) \log V)$  adj[u] list of (v,weight) adjacent to u.

int prim(int V, vector<vpi> &adj) {
    priority_queue<pii, vector<pii>, greater<pii>> q;
    vector<bool> vis(V, false);
    q.push({0, 0});
    int res = 0;
    while(!q.empty()){
        auto [w,u] = q.top();q.pop();
        if(vis[u])continue;
        res += w;
        vis[u] = true;
        for(auto v : adj[u]){
            if(!vis[v.first]){
                q.push({v.second, v.first});
            }
        }
    }
    return res;
}

```

2.12 Shortest Path

Find shortest paths from src (no negative weights). $O((V+E)\log V)$

```

vi dijkstra(const vector<vector<pii>>& adj, int src) {
    vi dist(adj.size(), INT_MAX);
    priority_queue<pii, vector<pii>, greater<pii>> q;
    dist[src] = 0; q.push({0, src});
    while (!q.empty()) {
        auto [d, u] = q.top(); q.pop();
        if (d != dist[u]) continue;
        for (auto [v, w] : adj[u]) {
            if (d+w < dist[v]) {
                dist[v] = d+w;
                q.push({d+w, v});
            }
        }
    }
    return dist;
}

```

$O(VE)$ Shortest Path+neg edges; BFS from nodes with dist $-\infty$ for all neg-cycle reachable.

```

vi bellmanFord(int n, vvi& edges, int src) {
    vector<int> dist(n, INT_MAX);
    dist[src] = 0;
    for (int i = 0; i < n; i++) {
        for (auto edge : edges) {
            int u = edge[0];int v = edge[1];int wt = edge[2];
            if (dist[u] != INT_MAX && dist[u] + wt < dist[v]) {
                if(i == n - 1) return {-1};
                dist[v] = dist[u] + wt;
            }
        }
    }
    return dist;
}

```

All-pairs shortest paths (neg edges ok, no neg cycles) $O(V^3)$ $graph[i][i] = 0$, $graph[i][j] = w$ if edge $i \rightarrow j$ else INT_MAX

```
vector<vi> floydWarshall(vector<vi> graph) {
    int V = graph.size();
    auto dist = graph;
    for (int k = 0; k < V; ++k)
        for (int i = 0; i < V; ++i)
            for (int j = 0; j < V; ++j)
                if (dist[i][k] < INT_MAX && dist[k][j] < INT_MAX)
                    dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]);

    return dist;
}
```

2.13 Strongly Connected Components

Kosaraju SCC $O(n+m)$

```
struct SCC {
    int n, ordn = 0, scc_cnt = 0;
    vvi adj, adjt; vi vis, ord, scc;
    SCC(int n): n(n), adj(n), adjt(n), vis(n,0), ord(n), scc(n) {}
    void add_edge(int u, int v){ adj[u].push_back(v); adjt[v].push_back(u); }
    void dfs(int u){
        vis[u]=1;
        for(int v:adj[u]) if(!vis[v]) dfs(v);
        ord[ordn++]=u;
    }
    void dfst(int u){
        scc[u]=scc_cnt; vis[u]=0; // SCCs IDs are in reverse topological order.
        for(int v:adjt[u]) if(vis[v]) dfst(v);
    }
    void kosaraju(){
        for(int i=0;i<n;i++) if(!vis[i]) dfs(i);
        for(int i=ordn-1;i>=0;i--) if(vis[ord[i]]){ scc_cnt++; dfst(ord[i]); }
    }
};
```

2.14 Toposort

TopoSort via DFS $O(V + E)$.

```
void dfs(int u, vector<vi> &adj, vi &vis, vi &res) {
    vis[u] = 1;
    for (int v : adj[u])
        if (!vis[v])
            dfs(v, adj, vis, res);
    res.push_back(u);
}

vi toposort(vector<vi> &adj) {
    int n = adj.size();
    vi vis(n, 0), res;
    for (int i = 0; i < n; i++)
        if (!vis[i]) dfs(i, adj, vis, res);
    reverse(res.begin(), res.end());
    return res;
}
```

3 Dynamic Programming

3.1 Coin Change

Given coins $c_1, \dots, c_k$ and a sum S :

Min coins to make S : DP Formula: $dp[x]$: min coins to make x , $dp[0] = 0$. $dp[x] = \min_{c_i \leq x} (dp[x - c_i] + 1)$. Ans = $dp[S]$.

Number of ways to make S : DP formula: $dp[x]$: #ways to make x , $dp[0] = 1$. $dp[x] = \sum_{c_i \leq x} dp[x - c_i]$. Ans = $dp[S]$.

Number of ways to make S , $O(n \times \text{sum})$

```
def count(coins, s):
    n = len(coins) ; dp = [0] * (s + 1) ; dp[0] = 1
    for i in range(n):
        for j in range(coins[i], s + 1):
            dp[j] += dp[j - coins[i]]
    return dp[s]
```

3.2 Convex Hull Trick

Maintains lines $y = m \cdot x + b$, supports $\text{add}(m, b)$ and $\text{query}(x)$ for max y at given x . Works in $O(\log n)$ per operation using a multiset and integer intersections.

```
struct Line {
    mutable ll m, b, p;
    bool operator<(const Line& o) const { return m < o.m; }
    bool operator<(ll x) const { return p < x; }
};

struct LineContainer : multiset<Line, less<>> {
    const ll inf = LLONG_MAX;
    ll div(ll a, ll b) {
        return a / b - ((a ^ b) < 0 && a % b);
    }
    bool isect(iterator x, iterator y) {
        if (y == end()) { x->p = inf; return false; }
        if (x->m == y->m) x->p = x->b > y->b ? inf : -inf;
        else x->p = div(y->b - x->b, x->m - y->m);
        return x->p >= y->p;
    }
    void add(ll m, ll b) {
        auto z = insert({m, b, 0}), y = z++, x = y;
        while (isect(y, z)) z = erase(z);
        if (x != begin() && isect(--x, y)) isect(x, y = erase(y));
        while ((y = x) != begin() && (--x)->p >= y->p) isect(x, erase(y));
    }
    ll query(ll x) {
        assert(!empty());
        auto l = *lower_bound(x);
        return l.m * x + l.b;
    }
};
```

3.3 Kadane

Given an array $a_1, \dots, a_n$, find $\max_{1 \leq i \leq j \leq n} \sum_{k=i}^j a_k$

DP formula: dp_i : max sum of subarray ending at i , $dp_i = \max(a_i, dp_{i-1} + a_i)$, Ans = $\max_i dp_i$.

Kadane 1D $O(n)$

```
def kadane1D(arr):
    maxSum = arr[0] ; DP = arr[0]
    for i in range(1, len(arr)):
        DP = max(DP + arr[i], arr[i])
        maxSum = max(maxSum, DP)
    return maxSum
```

2D-version : For $r_1 \leq r_2$: $\text{colSum}[c] = \sum_{r=r_1}^{r_2} A_{r,c}$, $\text{maxSum} = \max(\text{maxSum}, \text{Kadane1D}(\text{colSum}))$.

Kadane 2D $O(n^3)$

```
def kadane2D(mat):
    rows = len(mat) ; cols = len(mat[0]) ; maxSum = float('-inf') ; temp = [0] * cols
    for left in range(cols):
        temp = [0] * rows
        for right in range(left, cols):
            for row in range(rows):
                temp[row] += mat[row][right]
            sumValue = kadane1D(temp)
            maxSum = max(maxSum, sumValue)
```



```
return maxSum
```

3.4 Knapsack

n items, each with value v_i and weight w_i . Max total weight W . DP formula: $dp[i][j]$ max value using i items, where max total weight is j . $dp[i][w] = \max(dp[i-1][w], dp[i-1][w-w_i] + v_i)$. Space optimized version $O(n \times W)$

```
def knapsack(W, val, wt):
    dp = [0] * (W + 1)
    for i in range(1, len(wt) + 1):
        for j in range(W, wt[i - 1] - 1, -1):
            dp[j] = max(dp[j], dp[j - wt[i - 1]] + val[i - 1])
    return dp[W]
```

3.5 Longest Increasing Subsequence

ℓ_i : LIS length ending at i . DP formula : $\ell_i = 1 + \max_{1 \leq j < i \atop a_j < a_i} \ell_j$ (or 1 if none). Ans = $\max_i \ell_i$.

Predecessor $p_i = \arg \max_{1 \leq j < i \atop a_j < a_i} \ell_j$.

$O(n^2)$

```
def LIS(arr):
    n = len(arr) ; lis = [1] * n
    for i in range(1, n):
        for prev in range(0, i):
            if arr[i] > arr[prev]:
                lis[i] = max(lis[i], lis[prev] + 1)
    return max(lis)
```

Note that $O(n \log n)$ is possible with BS

```
def LIS_BS(arr):
    n = len(arr) ; ans = [] ; ans.append(arr[0])
    for i in range(1, n):
        if arr[i] > ans[-1]:
            ans.append(arr[i])
        else:
            low = 0
            high = len(ans) - 1
            while low < high:
                mid = low + (high - low) // 2
                if ans[mid] < arr[i]:
                    low = mid + 1
            else:
                high = mid
            ans[low] = arr[i]
    return len(ans)
```

3.6 MinPathSum

Given a grid with non negative ints, where you can only move right/down finds the min sum path from top-left to bottom-right.

$O(n \times m)$ Modifies grid.

```
11 minPathSum(vvi &grid){
    for (int i = 0; i < grid.size(); i++){
        for (int j = 0; j < grid[0].size(); j++){
            if (i == 0 && j == 0) continue;
            if (i == 0) grid[i][j] += grid[i][j - 1];
            else if (j == 0) grid[i][j] += grid[i - 1][j];
            else grid[i][j] += min(grid[i - 1][j], grid[i][j - 1]);
        }
    }
    return grid[grid.size() - 1][grid[0].size() - 1];
}
```

3.7 TSP

Traveling Salesman Problem $O(n^2 2^n)$ DP formula: $dp[S][i]$: min cost to start at 0, visit all $v \in S$, end at i . $dp[\{0\}][0] = 0$. $dp[S][i] = \min_{j \in S \setminus \{i\}} dp[S \setminus \{i\}][j] + w_{j,i}$. Ans = $\min_i dp[V][i] + w_{i,0}$.

```
int tsp(int n, vvi& dist) {
    int mask_limit = 1 << n;
    vvi dp(mask_limit, vi(n, INT_MAX));
    dp[1][0] = 0;
    for (int mask = 1; mask < mask_limit; mask++) {
        for (int last = 0; last < n; last++) {
            if (dp[mask][last] == INT_MAX) continue;
            for (int next = 0; next < n; next++) {
                if (mask & (1 << next)) continue;
                int new_mask = mask | (1 << next);
                dp[new_mask][next] = min(dp[new_mask][next],
                                         dp[mask][last] + dist[last][next]);
            }
        }
    }
    int ans = INT_MAX;
    for (int last = 1; last < n; last++) {
        if (dp[mask_limit - 1][last] != INT_MAX && dist[last][0] != INT_MAX) {
            ans = min(ans, dp[mask_limit - 1][last] + dist[last][0]);
        }
    }
    return ans;
}
```

4 Strings

4.1 Aho-Corasick

Finds the occurrences of a set of patterns in a text simultaneously $O(n + m + matches)$, where $n = \text{len}(\text{text})$, $m = \text{total pattern length}$.

```
class TrieNode:
    def __init__(self):
        self.children = {}
        self.output = []
        self.fail = None
def build_automaton(keywords):
    root = TrieNode()
    for keyword in keywords:
        node = root
        for char in keyword:
            node = node.children.setdefault(char, TrieNode())
        node.output.append(keyword)
    queue = deque()
    for node in root.children.values():
        node.fail = root
        queue.append(node)
    while queue:
        current = queue.popleft()
        for ch, nxt in current.children.items():
            queue.append(nxt)
            f = current.fail
            while f and ch not in f.children:
                f = f.fail
            nxt.fail = f.children[ch] if f else root
            nxt.output += nxt.fail.output
    return root
def search_text(text, keywords):
    root = build_automaton(keywords)
    res = {kw: [] for kw in keywords}
    node = root
```

```

for i, ch in enumerate(text):
    while node and ch not in node.children:
        node = node.fail
    if not node:
        node = root
        continue
    node = node.children[ch]
for kw in node.output:
    res[kw].append(i - len(kw) + 1)
return res

```

4.2 Knuth-Morris-Pratt

Search for a pattern within a text in $O(n + m)$, where n is the size of the text and m the length of the pattern. Length of the longest proper prefix which is also a suffix of the substring ending at that position.

```

def constructLps(pat, lps):
    len_ = 0 ; m = len(pat) ; lps[0] = 0 ; i = 1
    while i < m:
        if pat[i] == pat[len_]:
            len_ += 1
            lps[i] = len_
            i += 1
        else:
            if len_ != 0:
                len_ = lps[len_ - 1]
            else:
                lps[i] = 0
                i += 1
def search(pat, txt):
    n = len(txt) ; m = len(pat) ; lps = [0] * m ; res = []
    constructLps(pat, lps)
    i = 0 ; j = 0
    while i < n:
        if txt[i] == pat[j]:
            i += 1 # If characters match, move both pointers forward
            j += 1
            if j == m: #found
                res.append(i - j)
                j = lps[j - 1] # Use LPS to skip unnecessary comparisons
        else: # mismatch
            if j != 0:
                j = lps[j - 1] # Use LPS to skip unnecessary comparisons
            else:
                i += 1
    return res

```

4.3 Manacher

Longest palindromic substring $O(n)$. Max of P corresponds to $\text{len}(LPS)$

```

def manacher(s):
    T = '#' + '#' + s + '#' # handle even-length, remember to remove after
    n = len(T)
    P = [0]*n # P[i] = radius of palindrome centered at i
    C = R = 0 # current center, right boundary
    for i in range(n):
        mirr = 2*C - i
        if i < R: P[i] = min(R-i, P[mirr]) # use symmetry
        while i+P[i]+1 < n and i-P[i]-1 >= 0 and T[i+P[i]+1] == T[i-P[i]-1]:
            P[i] += 1 # attempt to expand palindrome at i
        if i + P[i] > R: # update center and right boundary
            C, R = i, i + P[i]

```

4.4 SuffixArray

SuffixArray i.e. starting positions of suffixes in lexicographic order. $O(n \log^2 n)$

```

def build_sa(s):
    n = len(s)
    sa = list(range(n))
    rank = [ord(ch) for ch in s] # initial ranks by 1 char
    k = 1
    while k < n: # sort by first k chars
        sa.sort(key=lambda x: (rank[x], rank[x+k] if x+k < n else -1))
        temp = [0]*n
        temp[sa[0]] = 0
        for i in range(1, n):
            prev = sa[i-1]
            cur = sa[i]
            prev_key = (rank[prev], rank[prev+k] if prev+k < n else -1)
            cur_key = (rank[cur], rank[cur+k] if cur+k < n else -1)
            temp[cur] = temp[prev] + (cur_key > prev_key)
        rank = temp
        if rank[sa[-1]] == n-1: break # all ranks distinct
        k <= 1
    return sa

```

Longest Common Prefix between suffixes $sa[i]$ and $sa[i+1]$ $O(n)$

```

def build_lcp(s, sa):
    n = len(s)
    rank = [0]*n
    for i in range(n): rank[sa[i]] = i
    lcp = [0]*n
    k = 0
    for i in range(n):
        if rank[i] == n-1: k = 0; continue
        j = sa[rank[i] + 1]
        while i+k < n and j+k < n and s[i+k] == s[j+k]:
            k += 1
        lcp[rank[i]] = k
        if k: k -= 1
    return lcp

```

Longest Repeated Substring = max LCP over all adjacent suffixes.

```

def lrs(s):
    sa = build_sa(s)
    lcp = build_lcp(s, sa)
    idx = lcp.index(max(lcp))
    return (lcp[idx], idx) # (length, position in SA)

```

Finds all occurrences of pat in s via binary search on the SA. Returns $[L, R]$: all SA positions where suffix starts with pat .

```

def search(s, pat, sa):
    n, m = len(s), len(pat)
    l, r = 0, n-1
    while l <= r:
        mid = (l+r)//2
        if s[sa[mid]:sa[mid]+m] < pat: l = mid+1
        else: r = mid-1
    start = l
    l, r = 0, n-1
    while l <= r:
        mid = (l+r)//2
        if s[sa[mid]:sa[mid]+m] <= pat: l = mid+1
        else: r = mid-1
    return (start, r) if start <= r else (-1, -1)

```

Longest Common Substring between two strings. Build SA/LCP on $s1\#s2\$, then the LCS occurs between adjacent suffixes coming from different strings.$

```
def lcs(s1, s2):
    s = s1 + '#' + s2 + '$'
    sa = build_sa(s)
    lcp = build_lcp(s, sa)
    idx, max_len = 0, 0
    for i in range(1, len(s)):
        a = sa[i] < len(s1)
        b = sa[i-1] < len(s1)
        if a != b and lcp[i] > max_len:
            max_len, idx = lcp[i], i
    return (max_len, idx)
```

5 Arrays

5.1 Inversions

Count pairs where order flips between arrays. $O(n \log n)$

```
11 inversions(vi& a, vi& b) {
    int n = a.size();
    unordered_map<int,int> pos;
    for (int i = 0; i < n; i++) pos[b[i]] = i + 1;
    Fenwick t(n); // C.f. Fenwick tree
    ll inv = 0;
    for (int i = 0; i < n; i++) {
        inv += i - t.sum(pos[a[i]]);
        t.add(pos[a[i]], 1);
    }
    return inv;
}
```

6 Math

6.1 Fourier Transform

FFT: in-place, invert=false for forward, true for inverse Input size must be power of two $O(n \log n)$

```
void fft(vector<complex<double>> &a, bool invert) {
    int n = a.size();
    for (int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1)
            j ^= bit;
        j ^= bit;
        if (i < j) swap(a[i], a[j]);
    }
    for (int len = 2; len <= n; len <= 1) {
        double ang = 2 * M_PI / len * (invert ? -1 : 1);
        complex<double> wlen(cos(ang), sin(ang));
        for (int i = 0; i < n; i += len) {
            complex<double> w(1);
            for (int j = 0; j < len / 2; j++) {
                complex<double> u = a[i + j];
                complex<double> v = a[i + j + len / 2] * w;
                a[i + j] = u + v;
                a[i + j + len / 2] = u - v;
                w *= wlen;
            }
        }
    }
    if (invert) {
        for (auto &x : a) x /= n;
    }
}
```

Multiply two polynomials, where $a[i]$ is the coefficient of x^i

```
vi multiply(vi &a, vi &b) {
    vector<complex<double>> fa(a.begin(), a.end()), fb(b.begin(), b.end());
    int n = 1; while (n < a.size() + b.size()) n <= 1; // next power of 2
    fa.resize(n); fb.resize(n);
    fft(fa, false); fft(fb, false); // forward FFT
    for (int i = 0; i < n; i++) fa[i] *= fb[i]; // pointwise multiply
    fft(fa, true); // inverse FFT
    vi res(n);
    for (int i = 0; i < n; i++) res[i] = round(fa[i].real());
    return res;
}
```

6.2 Linear Algebra

Solve $Ax = b$ using Gaussian elimination $O(n^3)$

```
def solve_gauss(a, b):
    n = len(a)
    a = [row[:] + [b[i]] for i, row in enumerate(a)]
    for i in range(n):
        pivot_row = max(range(i, n), key=lambda r: abs(a[r][i]))
        if abs(a[pivot_row][i]) < 1e-12:
            return None
        a[i], a[pivot_row] = a[pivot_row], a[i]
        pivot = a[i][i]
        for j in range(i, n + 1):
            a[i][j] /= pivot
        for r in range(i + 1, n):
            factor = a[r][i]
            for c in range(i, n + 1):
                a[r][c] -= factor * a[i][c]

    x = [0] * n
    for i in range(n - 1, -1, -1):
        x[i] = a[i][n]
        for j in range(i + 1, n):
            x[i] -= a[i][j] * x[j]

    return x
```

Maximizes $c^T x$ s.t. $Ax \leq b, x \geq 0$

```
def simplex(A, b, c):
    m, n = len(A), len(c)
    tableau = []
    for i in range(m):
        tableau.append(A[i][:] + [1 if j == i else 0 for j in range(m)] + [b[i]])
    tableau.append([-c_j for c_j in c] + [0] * (m + 1))
    basic = list(range(n, n + m))
    while True:
        entering = -1
        for j in range(n + m):
            if tableau[-1][j] < -1e-12:
                if entering == -1 or tableau[-1][j] < tableau[-1][entering]:
                    entering = j
        if entering == -1: # Optimal solution found
            break
        leaving = -1
        min_ratio = float('inf')
        for i in range(m):
            if tableau[i][entering] > 1e-12:
                ratio = tableau[i][-1] / tableau[i][entering]
                if ratio < min_ratio - 1e-12:
                    min_ratio = ratio
                    leaving = i
        if leaving == -1: # Unbounded problem
            return None, float('inf')
```

```

pivot = tableau[leaving][entering]
for j in range(n + m + 1):
    tableau[leaving][j] /= pivot
for i in range(m + 1):
    if i != leaving:
        factor = tableau[i][entering]
        for j in range(n + m + 1):
            tableau[i][j] -= factor * tableau[leaving][j]
basic[leaving] = entering
x = [0] * n
for i in range(m):
    if basic[i] < n:
        x[basic[i]] = tableau[i][-1]
return x, tableau[-1][-1]

```

6.3 Matrix Operations

Performs matrix multiplication in $O(n^3)$. Watch out for overflow.

```

matrix mult(const matrix &a, matrix &b) {
    int i, j, k, m = a.size(), n = a[0].size(), p = b[0].size();
    matrix res = matrix(m, vi(p, 0));
    for (i = 0; i < m; i++) for (j = 0; j < p; j++)
        for (k = 0; k < n; k++)
            res[i][j] += a[i][k] * b[k][j];
    return res;
}

```

Calculates a^p in $O(n^3 \log p)$. Watch out for overflow. Modifies matrix.

```

matrix pow(matrix &a, int p) {
    int l = a.size(), i;
    matrix b(l, vi(l, 0));
    for (i = 0; i < l; i++) b[i][i] = 1;
    while (p) {
        if (p & 1) {b = mult(b, a); p--;}
        a = mult(a, a);
        p >>= 1;
    }
    return b;
}

```

Gauss-Jordan on an augmented matrix, returns determinant, modifies matrix. $O(n^3)$

```

double gaussJordan(matrix &a, int &rank, int &augRank) {
    int n = a.size(), m = a[0].size() - 1; // n eqs, m vars
    double det = 1.0; rank = 0;
    for (int j = 0; j <= m; j++) {
        int pivotRow = -1;
        double maxVal = 0;
        for (int i = rank; i < n; i++)
            if (pivotRow < 0 || abs(a[i][j]) > maxVal) {
                maxVal = abs(a[i][j]);
                pivotRow = i;
            }
        if (maxVal > 1e-7) {
            double c = a[pivotRow][j];
            if (pivotRow != rank) det = -det;
            det /= c;
            for (int k = j; k <= m; k++) {
                swap(a[pivotRow][k], a[rank][k]);
                a[rank][k] /= c;
            }
            for (int i = rank + 1; i < n; i++) {
                c = a[i][j];
                for (int k = j; k <= m; k++)
                    a[i][k] -= c * a[rank][k];
            }
        }
    }
}

```

```

        if (j < m) rank++;
    }
    augRank = rank;
    for (int i = rank; i < n; i++)
        if (abs(a[i][m]) > 1e-6) { augRank++; break; }
    if (rank != n) det = 0;
    return det;
}

```

Solve $Ax=b$. Input: $[A|b]$ (augmented matrix) $O(n^3)$. Modifies Matrix.

```

void solve(matrix &a, vi &solution) {
    int n = a.size(), m = a[0].size() - 1;
    solution.assign(m, 0.0);
    for (int i = n - 1; i >= 0; i--) {
        double c = a[i][m];
        for (int j = i + 1; j < m; j++)
            c -= a[i][j] * solution[j];
        solution[i] = c;
    }
}

```

6.4 Number Theory

Linear diophantine equations: Let $a, b, c \in \mathbb{N}$. The equation $ax + by = c$ has integer solutions iff c is a multiple of $\gcd(a, b)$. If (x_0, y_0) is a solution for the equation (c.f. Extended Euclidean Algorithm), then all integer solutions are given by $(x_0 - b't, y_0 + a't)$ with $t \in \mathbb{N}$, $a' = \frac{a}{\gcd(a, b)}$, $b' = \frac{b}{\gcd(a, b)}$.

Chinese remainder theorem: Let $n_1, \dots, n_k \in \mathbb{N}$ pairwise coprime, and $b_1, \dots, b_k \in \mathbb{N}$. The system of linear congruencies $x \equiv b_i \pmod{n_i}$ for $1 \leq i \leq k$ has a unique solution $\pmod{\prod_{i=1}^k n_i}$, constructed as follows: Let $a_i = \frac{N}{n_i}$ and c_i its inverse $\pmod{n_i}$. Then, $x \equiv \sum_{i=1}^k a_i b_i c_i \pmod{\prod_{i=1}^k n_i}$

$\gcd(a, b) = \gcd(a, b \bmod a)$, $\text{lcm}(a, b) = \frac{ab}{\gcd(a, b)}$, $\gcd(a, b, c) = \gcd(\gcd(a, b), c)$, $\text{lcm}(a, b, c) = \text{lcm}(\text{lcm}(a, b), c)$

6.5 Useful NT Functions in Python

Sieve of Gries-Misra $O(n)$. Produces prime list and SPF array

```

def gries_misra(n):
    primes = []
    factor = [0] * n
    for x in range(2, n):
        if not factor[x]: # no factor found
            factor[x] = x # meaning x is prime
            primes.append(x)
        for p in primes: # loop over primes found so far
            if p > factor[x] or p * x >= n:
                break
            factor[p * x] = p # p is the smallest factor of p * x
    return primes, factor

```

Returns (u, v) s.t. $au + bv = \gcd(a, b)$ in $O(\log a + \log b)$

```

def bezout(a, b):
    if b == 0:
        return (1, 0)
    u, v = bezout(b, a % b)
    return (v, u - (a // b) * v)

```

Inverse of a in $\mathbb{Z}_p$ in $O(\log a + \log p)$

```

def inv(a, p):
    return bezout(a, p)[0] % p

```

$\binom{n}{k} \pmod p$ in $O(k)$

```
def binom_modulo(n, k, p):
    prod = 1
    for i in range(k):
        prod = (prod * (n - i) * inv(i + 1, p)) % p
    return prod
```

Constructs an arithmetic expression tree. Input: tokens e.g. ["1", "+", "2"]

```
def arithm_expr_parse(tokens):
    vals, ops = [], []
    PRIO = {'+':1, '-':1, '*':2, '/':2, '(':0}
    for t in tokens:
        if t in PRIO:
            while ops and PRIO[ops[-1]] >= PRIO[t] and t != '(':
                vals.append((vals.pop(-2), ops.pop(), vals.pop()))
                ops.append(t)
            elif t == ')':
                while ops[-1] != '(':
                    vals.append((vals.pop(-2), ops.pop(), vals.pop()))
                    ops.pop()
            else:
                vals.append(int(t) if t.isdigit() else t)
    while ops:
        vals.append((vals.pop(-2), ops.pop(), vals.pop()))
    return vals[0]
```

6.6 Useful NT Functions in c++

Sieve of Eratosthenes: $O(n \log \log n)$ sieve[i] = 0 if i prime, spf otherwise for fast fact

```
vi sieve(int n){
    vi sieve(n,0);
    for(int i = 2; i*i < n ; i++){
        if(!sieve[i])
            for(int j = i*i ; j < n ; j += i)
                if(!sieve[j]) sieve[j] = i;
    }
    return sieve;
}
```

Fast exponentiation $O(\log b)$: fast $a^b \bmod p$.

```
int modular_exp(int a, int b, int p){
    int res = 1;
    while(b > 0){
        if(b & 1) res = (1LL * a * res) % p;
        b = b >> 1;
        a = (1LL * a * a) % p;
    }
    return res;
}
```

Euler's φ : Number of ints $\leq n$ coprime to n.

```
int phi(int n, vi& primes) {
    int res = n;
    for (int p : primes) {
        if (1LL * p * p > n) break;
        if (n % p == 0) {
            while (n % p == 0) n /= p;
            res -= res / p;
        }
    }
    if (n > 1) res -= res / n;
    return res;
}
```

Extended Euclidean Algorithm Solves $ax + by = \gcd(a, b)$.

```
ll eea(ll a, ll b, ll& x, ll& y) {
    ll xx = y = 0, yy = x = 1;
```

```
    while (b) {
        ll q = a / b, t = b; b = a%b; a = t;
        t = xx; xx = x - q*xx; x = t;
        t = yy; yy = y - q*yy; y = t;
    }
    return a; // Returns gcd(a,b)
}
```

7 Geometry

7.1 Circles

Retourne par reference le centre d'un cercle qui passe par 2 points et a un rayon donne

```
bool circle2PtsRad(point p1, point p2, double r, point &c) {
    double d2 = (p1.x - p2.x) * (p1.x - p2.x) +
                (p1.y - p2.y) * (p1.y - p2.y);
    double det = r * r / d2 - 0.25;
    if (det < 0.0) return false;
    double h = sqrt(det);
    c.x = (p1.x + p2.x) * 0.5 + (p1.y - p2.y) * h;
    c.y = (p1.y + p2.y) * 0.5 + (p2.x - p1.x) * h;
    return true;
}
```

Rayon du cercle inscrit donnees les longueurs des cotes

```
double rInCircle(double ab, double bc, double ca) {
    return heron(ab, bc, ca) / (0.5 * (ab+ bc+ca)); }
```

Retourne le centre et le rayon du cercle inscrit d'un triangle (par reference)

```
bool inCircle(point p1, point p2, point p3, point &ctr, double &r) {
    r = rInCircle(p1.dist(), p2.dist(), p3.dist());
    if (abs(r) < EPS) return false; // no inCircle center
    line l1, l2; // compute these two angle bisectors
    double ratio = dist(p1, p2) / dist(p1, p3);
    point p = translate(p2, scale(toVec(p2, p3), ratio / (1 + ratio)));
    pointsToLine(p1, p, l1);
    ratio = dist(p2, p1) / dist(p2, p3);
    p = translate(p1, scale(toVec(p1, p3), ratio / (1 + ratio)));
    pointsToLine(p2, p, l2);
    areIntersect(l1, l2, ctr); // get their intersection point
    return true;
}
```

Rayon du cercle circonscrit donnees les longueurs des cotes

```
double rCircumCircle(double ab, double bc, double ca) {
    return ab * bc * ca / (4.0 * heron(ab, bc, ca)); }
```

Retourne le centre et le rayon du cercle circonscrit d'un triangle (par reference)

```
bool circumCircle(point p1, point p2, point p3, point &ctr, double &r){
    double a = p2.x - p1.x, b = p2.y - p1.y, c = p3.x - p1.x, d = p3.y - p1.y;
    double e = a * (p1.x + p2.x) + b * (p1.y + p2.y);
    double f = c * (p1.x + p3.x) + d * (p1.y + p3.y);
    double g = 2.0 * (a * (p3.y - p2.y) - b * (p3.x - p2.x));
    if (fabs(g) < EPS) return false;
    ctr.x = (d*e - b*f) / g; ctr.y = (a*f - c*e) / g;
    r = dist(p1, ctr); // r = distance from center to 1 of the 3 points
    return true;
}
```

Retourne l'orthocentre d'un triangle. r n'a pas de sens ici

```
bool orthocenter(point a, point b, point c, point & ctr, double &r){
    return circumCircle(a+b-c, b+c-a, c+a-b, ctr, r);
}
```

7.2 Closest Points

Minimum Euclidean distance between two distinct points. $O(n \log n)$

```
vector<point> p;
double cp(int l, int r) {
    if (l >= r) return DBL_MAX;
    int mid = (l + r) / 2;
    double d = min(cp(l, mid), cp(mid + 1, r));
    vector<point> strip;
    for (int i = l; i <= r; i++)
        if (fabs(p[i].x - p[mid].x) <= d)
            strip.push_back(p[i]);
    sort(strip.begin(), strip.end(), [](point a, point b){return a.y < b.y;});
    for (int i = 0; i < strip.size(); i++)
        for (int j = i+1; j < strip.size() && (strip[j].y - strip[i].y) <= d; j++)
            d = min(d, dist(strip[i], strip[j]));
    return d;
}
```

7.3 Points and Lines

#define EPS 1e-6

Point template, use long long or double, avoid int.

```
template <class T>
struct Point {
    T x, y;
    explicit Point(T x=0, T y=0) : x(x), y(y) {}
    bool operator<(P p) const { return tie(x,y) < tie(p.x,p.y); }
    bool operator==(P p) const { return tie(x,y)==tie(p.x,p.y); }
    P operator+(P p) const { return P(x+p.x, y+p.y); }
    P operator-(P p) const { return P(x-p.x, y-p.y); }
    P operator*(T d) const { return P(x*d, y*d); }
    P operator/(T d) const { return P(x/d, y/d); }
    T dot(P p) const { return x*p.x + y*p.y; }
    T cross(P p) const { return x*p.y - y*p.x; }
    T cross(P a, P b) const { return (a-*this).cross(b-*this); }
    T dist2() const { return x*x + y*y; }
    double dist() const { return sqrt((double)dist2()); }
}
```

angle to x-axis in interval $[-\pi, \pi]$

```
double angle() const { return atan2(y, x); }
P unit() const { return *this/dist(); } // makes dist()=1
P perp() const { return P(-y, x); } // rotates +90 degrees
P normal() const { return perp().unit(); }
```

returns point rotated 'a' radians ccw around the origin

```
P rotate(double a) const {
    return P(x*cos(a)-y*sin(a),x*sin(a)+y*cos(a)); }
};
```

Vector $a \vec{b}$

```
point toVec(point a, point b) { return point(b.x-a.x, b.y-a.y); }
point scale(point v, double s) { return point(v.x*s, v.y*s);}
point translate(point p, point v) { return point(p.x + v.x, p.y + v.y);}
double cross(point a, point b, point c) { return toVec(a, b).cross(toVec(b, c)); }
double dist(point p1, point p2) { return hypot(p1.x - p2.x, p1.y - p2.y);}
double distSq(point p1, point p2) {
    return (p1.x - p2.x)*(p1.x - p2.x) + (p1.y - p2.y)*(p1.y - p2.y);
}
```

pour accepter des points colineaires remplacer $\vec{b}$ par $\vec{c}$ = et ajouter un EPS peut-etre returns true if point r is on the left side of line pq

```
bool ccw(point p, point q, point r) {
    return (toVec(p, q).cross(toVec(p, r))) > 0;
}
```

```
bool collinear(point p, point q, point r) {
    return abs(toVec(p, q).cross(toVec(p, r))) < EPS;
}
int insideCircle(point p, point c, double r) {
    double eucSq = distSq(p, c);
    double rSq = r*r;
    if (fabs(eucSq - rSq) > EPS && eucSq < rSq) return 0; // inside
    else if (fabs(eucSq - rSq) < EPS) return 1; // border
    else return 2; // outside
}
```

returns $\angle AOB$ in rad

```
double angle(point a, point o, point b) {
    point oa = toVec(o, a), ob = toVec(o, b);
    return acos(oa.dot(ob) / sqrt(oa.dist2() * ob.dist2()));
}
```

distance from p to the line defined by two points a and b. The closest point to p on the segment is stored in c.

```
double distToLine(point p, point a, point b, point &c) {
    point ap = toVec(a, p), ab = toVec(a, b);
    double u = ap.dot(ab) / ab.dist2();
    c = translate(a, scale(ab, u));
    return dist(p, c);
}
```

distance from p to the segment defined by two points a and b. The closest point to p on the segment is stored in c.

```
double distToSegment(point p, point a, point b, point &c) {
    auto ap = toVec(a, p), ab = toVec(a, b);
    double u = ap.dot(ab) / ab.dist2();
    if (u < 0.0) {
        c = point(a.x, a.y);
        return ap.dist();
    } else if (u > 1.0) {
        c = point(b.x, b.y);
        return toVec(p, b).dist();
    } else return distToLine(p, a, b, c);
}
```

Line representation using equation $ax + by + c = 0$

```
struct line { double a, b, c; };
```

Line that passes through two points, stored in the third parameter.

```
void pointsToLine(point p1, point p2, line &l) {
    if (fabs(p1.x - p2.x) < EPS) { // vertical
        l.a = 1.0; l.b = 0.0; l.c = -p1.x;
    } else {
        l.a = -(double)(p1.y - p2.y) / (p1.x - p2.x); l.b = 1.0;
        l.c = -(double)(l.a * p1.x) - p1.y;
    }
}
```

```
bool areParallel(line l1, line l2) { // check coefficients a & b
    return (fabs(l1.a-l2.a) < EPS) && (fabs(l1.b-l2.b) < EPS); }
```

Solves the linear system to find the intersection of two lines

```
bool areIntersect(line l1, line l2, point &p) {
    if (areParallel(l1, l2)) return false; // no intersection
    p.x = (l2.b * l1.c - l1.b * l2.c) / (l2.a * l1.b - l1.a * l2.b);
    if (fabs(l1.b) > EPS) p.y = -(l1.a * p.x + l1.c);
    else p.y = -(l2.a * p.x + l2.c);
    return true;
}
```

7.4 Polygons

#define PI 3.14159265358979323

Convex Hull


```
vector<point> andrew(vector<point> &P) {
    int n = P.size(), k = 0;
    vector<point> H(2*n);
    sort(P.begin(), P.end());
```

Build lower hull

```
    for (int i = 0; i < n; i++) {
        while (k >= 2 && !ccw(H[k-2], H[k-1], P[i])) k--;
        H[k++] = P[i];
    }
```

Build upper hull

```
    for (int i = n-2, t = k+1; i >= 0; i--) {
        while (k >= t && !ccw(H[k-2], H[k-1], P[i])) k--;
        H[k++] = P[i];
    }
    H.resize(k);
    return H;
}
```

```
double heron(double a, double b, double c){
    double s = (a+b+c)/2;
    return sqrt(s*(s-a)*(s-b)*(s-c));
}
double area(const vector<point> &P) {
    double result = 0.0, x1, y1, x2, y2;
    for (int i = 0; i < (int)P.size()-1; i++)
        result += (P[i].x * P[i+1].y - P[i+1].x * P[i].y);
    return abs(result) / 2.0; }
```

returns true if point p is in either convex/concave polygon P

```
bool inPolygon(point pt, const vector<point> &P) {
    if ((int)P.size() == 0) return false;
    double sum = 0;
    for (int i = 0; i < (int)P.size()-1; i++) {
        if (ccw(pt, P[i], P[i+1]))
            sum += angle(P[i], pt, P[i+1]);
        else sum -= angle(P[i], pt, P[i+1]); }
    return fabs(fabs(sum) - 2*PI) < EPS; }
```

$O(\log n)$ Seulement pour les convexes (ordonnes ccw)!

```
bool inConvexPolygon(point q, vector<point> const &p) {
    if (cross(p[0], p[1], q)<0 || cross(p[p.size()-2], p[0], q)<0)
        return false;
    int ini = 1, fin = p.size() - 2, mid;
    while (ini != fin - 1) {
        mid = (ini + fin) / 2;
        if (cross(p[0], p[mid], q) < 0) fin = mid;
        else ini = mid;
    }
    if (cross(p[ini], p[fin], q)<0) return false;
    return true;
}
```

Cut Polygon returns the point on segment PQ that is closest to line AB

```
point lineIntersectSeg(point p, point q, point A, point B) {
    double a = B.y - A.y, b = A.x - B.x, c = B.x * A.y - A.x * B.y;
    double u = abs(a * p.x + b * p.y + c), v = abs(a * q.x + b * q.y + c);
    return point((p.x * v + q.x * u) / (u+v), (p.y * v + q.y * u) / (u+v)); }
```

Cuts a polygon, keeping only the points on the left side of the directed line $a_i b$

```
vector<point> cutPolygon(point a, point b, const vector<point> &Q) {
    vector<point> P;
    for (int i = 0; i < (int)Q.size(); i++) {
        double left1 = toVec(a, b).cross(toVec(a, Q[i])), left2 = 0;
        if (i != (int)Q.size()-1) left2 = toVec(a, b).cross(toVec(a, Q[i+1]));
        if (left1 > -EPS) P.push_back(Q[i]); // Q[i] is on the left of ab
        if (left1 * left2 < -EPS) // edge (Q[i], Q[i+1]) crosses line ab
```

```
        P.push_back(lineIntersectSeg(Q[i], Q[i+1], a, b));
    }
    if (!P.empty() && !(P.back() == P.front()))
        P.push_back(P.front()); // make P's first point = P's last point
    return P; }
```

7.5 Useful Theory

8 Triangles

Triangle dont les côtés sont a, b, c , les hauteurs sont h_a, h_b, h_c , les médianes sont m_a, m_b, m_c , le demi-périmètre est $s = \frac{a+b+c}{2}$ et l'aire est A . $\eta = \frac{1}{4R}$

Heron-like area formulae

$$\begin{aligned} A &= \sqrt{s(s-a)(s-b)(s-c)} \\ &= \left(4\sqrt{\eta(\eta-h_a^{-1})(\eta-h_b^{-1})(\eta-h_c^{-1})}\right)^{-1} \\ &= \frac{4}{3}\sqrt{s(s-m_a)(s-m_b)(s-m_c)} \end{aligned}$$

Rayon du cercle circonscrit $R = \frac{abc}{4A}$

Rayon du cercle inscrit $r = \frac{A}{s}$

Médianes divisent en deux sous-triangles de même aire.

$$m_a = \frac{1}{2}\sqrt{2b^2 + 2c^2 - a^2}$$

Bissectrices divisent les angles en deux parties égales.

$$b_a = \sqrt{bc \left(1 - \left(\frac{a}{b+c}\right)^2\right)}$$

Loi des sinus $\frac{\sin \alpha}{a} = \frac{\sin \beta}{b} = \frac{\sin \gamma}{c} = \frac{1}{2R}$

Loi des cosinus $a^2 = b^2 + c^2 - 2bc \cos \alpha$

Théorème de la tangente $\frac{a+b}{a-b} = \frac{\tan \frac{\alpha+\beta}{2}}{\tan \frac{\alpha-\beta}{2}}$

9 Quadrilatères

9.1 Quadrilatères cycliques (inscriptibles)

Soit un quadrilatère **cyclique** de côtés a, b, c, d et diagonales e, f . Alors $ef = ac + bd$ et l'aire A peut être calculée grâce à la

Formule de Brahmagupta

$$A = \sqrt{(s-a)(s-b)(s-c)(s-d)}$$

$$\text{où } s = \frac{a+b+c+d}{2}$$

10 Coordonnées sphériques

$$\begin{aligned} x &= r \sin \theta \cos \phi & r &= \sqrt{x^2 + y^2 + z^2} \\ y &= r \sin \theta \sin \phi & \theta &= \arccos(z / \sqrt{x^2 + y^2 + z^2}) \\ z &= r \cos \theta & \phi &= \operatorname{atan2}(y, x) \end{aligned}$$

11 Utilities

11.1 STL

```
int main()
{
    Lambda functions

    int a;
    auto f = [](){};           // empty capture: no outer variables, only parameters +
                               ↪ globals
```

```
auto f = [=](int x){cout <<a;}; // capture all outer variables by value
auto f = [&](int x){a=2;};      // capture all outer variables by reference
```

Sort using custom function

```
vector<pair<int,int>> P = {{1,2},{3,0},{2,5}};
sort(P.begin(), P.end(), [](auto &a, auto &b){ return a.second < b.second; });
}
```